An Algebraic Framework for Financial Arbitrage Discovery in Decentralized Prediction Markets

bon-cdp

November 15, 2025

Abstract

This paper presents an algebraic framework, inspired by sheaf theory, for the direct and non-iterative discovery of arbitrage opportunities in decentralized prediction markets. We model market states as sections over a topological space of events, where local market prices are analogous to local sections. Arbitrage opportunities are then identified as "cohomological obstructions" – instances where local sections (market prices) cannot be consistently glued together to form a global, arbitrage-free section. Our approach leverages high-performance C++ implementations, including multithreading and SIMD, to achieve ultra-low latency detection, demonstrating that market inefficiencies can be precisely quantified and exploited through a purely algebraic lens.

1 Introduction

Decentralized prediction markets, built on blockchain technology, offer novel avenues for speculation and information aggregation. However, like traditional financial markets, they are susceptible to inefficiencies, particularly arbitrage opportunities where risk-free profit can be made by simultaneously buying and selling across different market offerings. Traditional machine learning approaches, often reliant on iterative gradient descent, are ill-suited for the real-time, exact nature of arbitrage discovery. This work proposes a novel algebraic framework for this problem.

2 A Sheaf-Theoretic Model of Prediction Markets

2.1 Definitions

We adapt concepts from sheaf theory to model the structure of prediction markets.

Definition 2.1 (Base Space of Events, X)**.** *Let X be a discrete topological space whose points $x \in X$ represent unique, underlying events on which predictions can be made (e.g., "Will SOL price be ¿ \$200 on Dec 1st, 2025?").*

Definition 2.2 (Patches/Open Sets, U_i)**.** *An open set $U_i \subseteq X$ corresponds to a specific prediction market provider or platform (e.g., Market A, Market B). Each U_i offers odds on a subset of events in X.*

Definition 2.3 (Sections, $s_i(x)$)**.** *A section $s_i(x)$ on an open set U_i (market i) for an event $x \in U_i$ is the set of prices offered by market i for the possible outcomes of event x. For a binary prediction market, this is typically a pair: $s_i(x) = \{price_{yes}, price_{no}\}$.*

2.2 The Gluing Condition and Cohomological Obstruction

In sheaf theory, sections on overlapping open sets must agree on their intersection (the "gluing condition") to form a global section. In our context, a "global section" would represent a perfectly efficient, arbitrage-free market state.

Definition 2.4 (Arbitrage as Cohomological Obstruction)**.** *For a given event $x \in X$, if two distinct markets U_i and U_j both offer prices for x, an arbitrage opportunity exists if the prices for complementary outcomes (e.g., 'yes' from market i and 'no' from market j) sum to less than 1.0. This represents a failure of the gluing condition. Specifically, for an event x:*

$$price_{yes}(U_i, x) + price_{no}(U_j, x) < 1.0$$

This inequality quantifies the "cohomological obstruction" to forming a globally consistent, arbitrage-free market state. The magnitude of $(1.0-(price_{yes}(U_i,x)+price_{no}(U_j,x)))$ represents the raw profit margin.

Remark 2.5. *Our solver's task is to efficiently identify all such instances of cohomological obstruction across the entire space of events and markets.*

3 The Solver as a Direct Algebraic Search

Unlike traditional machine learning models that iteratively learn parameters, our approach directly solves for these inconsistencies in a single, non-iterative pass.

[Arbitrage Discovery Algorithm] Given a stream of market outcomes ($EventID, MarketID, Price, IsYes$):

1. **Partitioning (Sheaf Construction):** Group all incoming market outcomes by their unique $EventID$. This effectively creates "stalks" of market data for each event $x \in X$.

2. **Local Section Analysis (Wreath Product Analogy):** For each $EventID$, identify the minimum $\text{price}_{\text{yes}}$ across all markets covering that event, and similarly the minimum price_{no}. This step implicitly considers the "position-dependent transformations" (pricing functions) of various markets.

3. **Gluing Condition Check (Obstruction Detection):** For each $EventID$, check if the sum of the globally minimum $\text{price}_{\text{yes}}$ and minimum price_{no} is less than 1.0. If so, an arbitrage opportunity (cohomological obstruction) is detected.

This process yields all arbitrage opportunities in a single, direct computation, without any iterative optimization.

4 High-Performance Implementation

To make this algebraic discovery practical for high-frequency trading, the solver is implemented in C++ with extreme performance considerations:

- **Multithreading (Thread Pool Batching):** The independent nature of event processing allows for massive parallelization. A custom thread pool processes events in large batches, minimizing thread creation/management overhead and maximizing CPU utilization.

- **SIMD (Single Instruction, Multiple Data):** Within each thread, the search for minimum prices for an event is accelerated using AVX2 intrinsics, allowing the CPU to perform operations on multiple floating-point numbers simultaneously.
- **Memory Efficiency:** Careful data structuring and use of move semantics minimize memory copies and maximize cache locality.

5 Conclusion

By reframing the problem of arbitrage discovery through a sheaf-theoretic lens, we transform it from an iterative learning task into a direct algebraic search for inconsistencies. This approach not only provides a clear theoretical foundation for market inefficiencies but also enables the development of ultra-low latency, high-performance C++ solutions capable of exploiting these opportunities in real-time decentralized prediction markets.